

# Signals Deep Dive

なぜ必要な箇所だけ更新できるのか

フロントエンドカンファレンス北海道 2026

nishitaku

# 自己紹介

- 西濃 拓郎（にしの たくろう） / @nishitaku
- フリーランス / フロントエンドエンジニア（Angular）
- 岐阜から来ました



## 話すこと

- Signals の本質は依存関係の追跡にある
- Signals がどうやって依存関係の追跡を実現しているか

## 話さないこと

- Signals の API を網羅的には紹介しません
- 各種FWの実装について

# Signals とは

- 状態と依存関係を扱うリアクティブモデル
- fine-grained reactivity を実現
- Angular / Vue / Solid / Preact / Qwik など採用
- TC39 proposal-signals (Stage 1)

```
let counter = 0;
const setCounter = (value) => {
  counter = value;
  renderParity();
}

const parity = () => (counter % 2) === 0 ? "even" : "odd";
const renderParity = () => element.innerText = parity();

setInterval(() => setCounter(counter + 1), 1000);
```

```
let counter = 0;
const setCounter = (value) => {
  counter = value;
  renderParity();

  // counterが変更された時に、colorも変更されることを知っている必要がある
  renderColor();
}

const parity = () => (counter % 2) === 0 ? "even" : "odd";
const renderParity = () => element.innerText = parity();

const color = () => (counter % 2) === 0 ? "blue" : "red";
const renderColor = () => counterEl.innerText = color();

setInterval(() => setCounter(counter + 1), 1000);
```

```
const counter = signal(0);

const parity = computed(() => (counter() % 2) == 0 ? "even" : "odd");

effect(() => {
  element.innerText = parity();
});

setInterval(() => counter.set(counter() + 1), 1000);
```

```
const name = "太郎";  
const age = 20;  
  
const validatedName = () => `${f(name)}`;  
const isEven = () => `${age} % 2 === 0 ? '偶数' : '奇数'`;
```

```
<p>名前は{{ validatedName }}です</p>  
<p>年齢は{{ isEven }}です。</p>
```



```
const name = "太郎";  
const age = 21; // 変更された場合  
  
const validatedName = () => `${f(name)}`;  
const isEven = () => `${age} % 2 === 0 ? '偶数' : '奇数'`; // 再計算したい
```

```
<p>名前は{{ validatedName }}です</p>  
<p>年齢は{{ isEven }}です。</p> // 更新したい
```

```
const name = signal("太郎");  
const age = signal(21);  
  
const validatedName = computed(() => `${f(name)}`);  
const isEven = computed(() => `${age} % 2 === 0 ? '偶数' : '奇数'`);
```

<p>名前は{{ validatedName() }}です</p>  
<p>年齢は{{ isEven() }}です。</p>

## Signals のメリット

- 状態間の依存関係を意識する必要がなくなる
- 依存している計算だけが行われるため、パフォーマンスが上がる

# Runtime Dependency Tracking

- ビルド時ではなく、実行時に依存を解決している

```
const value = computed(() => {  
  if (flag()) {  
    return count()  
  }  
  
  return another()  
})
```

# static vs runtime

## static tracking

- compile-time
- 速い
- runtime軽い
- dynamic dependency苦手

## runtime tracking

- 実行時tracking
- dynamic dependency OK
- fine-grained
- runtime graph必要

# Signals は observer pattern ?

- 広義のobserver pattern を含んでいる
- しかし本質は dependency graph
- Signalsは「読むことで依存登録」が重要
- computed による派生状態を扱える
- Push/Pull Hybrid によって lazy に再計算する

**Push-based or Pull-based ?**

# Push-based

状態変更時に、依存先へ即座に通知・再計算する方式。

例：Observer Pattern、Pub/Sub、EventEmitter、RxJS

メリット

- 常に最新状態
- 読み取り時は速い
- モデルが分かりやすい

デメリット

- 使われない値まで再計算
- update storm が起きやすい
- dependency graph が大きいと propagation コストが高い



# signals のデメリット

## グリッチフリー vs lossy

Q: 「グリッチフリー（不整合のない）」実行とはどういう意味ですか？

A: 初期のプッシュ型リアクティブモデルでは、State が変更されるたびに Computed が即座に走り、UI を更新しようとしていました。しかし、次のフレームまでに複数の変更がある場合、中間の不正確な値が表示される「グリッチ」が発生することがありました。Signal はプル型を採用することで、フレームワークが UI を描画するタイミングで必要な更新だけを取りに行くため、無駄な計算や DOM 操作、そして表示の不整合を避けることができます。

Q: 「損失がある（lossy）」とはどういう意味ですか？

A: これはグリッチフリーの裏返しです。Signal はデータの「セル」であり、現在の最新値を表します。時間の経過に伴う「ストリーム」ではありません。State に2回連続

# Agenda

1. Signalsとは何か
2. 依存関係はどう追跡されるのか
3. なぜ効率的なのか
4. signal-polyfillを触って見えたこと
5. まとめ

# 私とSignalsとの出会い

- Angular と zone.js
- v16 で 導入された Signals によって環境が激変
- Signals の魔法に感動して、仕組みが知りたくなった

## signal-polyfillを触ってみて

- 思ったよりシンプル
- 魔法ではない
- dependency graph の伝播アルゴリズム

## まとめ

- Signalsは dependency graph を構築する
- dependency tracking によって依存関係を収集する
- Push/Pull Hybrid により効率的に更新する

**Thank you**